

SLICE 14 — BUILD: stock enforcement + post-sale reset

Owner: Michael Lyman, Greenway Marijuana (Port Orchard, WA) **Status:** built, tested, shipped **Predecessor:** docs/slice-13-recon-stock-and-reset.md (recon only, no code edits)

Standing rule honoured throughout: *do not guess, do not assume — we build from fact, not memory*. Every line number in this document was re-read from the working tree with `grep -n` immediately before it was written down. Every regression test in this slice was demonstrated FAILING against the pre-fix code before the fix was applied, so we know the tests test the bug and not merely themselves.

1. What the owner reported

Two defects, found while testing the register at the counter.

Bug A — stock quantities are not respected. In the owner's own words: *"a product tile said '1 left', so I tapped it twice from the menu grid, and it let me add both times. I then went to the cart and used the plus minus buttons and they too let me add as many as I want."* He was explicit that the legal per-category limits DO work correctly; only the on-hand count was unenforced.

Bug B — the next customer inherits the last one's sale. *"After completing a sale, the register locks as it should. When I enter my pin to open it back up, it bypasses the age gate and loads the same products in the cart for the same customer."*

Bug B is the more serious of the two. Bug A promises product that may not be on the shelf. Bug B can attach one customer's verified ID to another customer's purchase, which is a compliance problem, not merely an inventory problem.

2. The four decisions that shaped the build

The recon slice ended with four open questions. The owner answered all four and authorised the build; the answers are recorded here because they are design constraints, not preferences, and anyone changing this code later needs them.

Out-of-stock tiles are greyed out AND explain themselves when tapped. The owner asked for the combination rather than either alone. This matters more than it sounds: a tile that is greyed out and also inert teaches staff that the register is broken, because a tap that produces no response is indistinguishable from a frozen screen. The tile is therefore visually de-emphasised but remains fully tappable, and tapping produces a plain-English refusal naming the product.

Any member of staff may override a stock block; every other override stays manager-locked. This is a deliberate asymmetry and it is the right one. A stock discrepancy is a counting error — the unit is either physically on the shelf or it is not, and the person holding it is the best judge. A price override or a scan-required bypass is a money-and-compliance decision and remains behind the manager PIN exactly as before. Nothing in this slice touched `price-override-core` or `scan-required-core`.

Block at tender, offer an override, and show when the menu was last refreshed. The owner accepted the recommendation from recon. The last-refreshed timestamp is the crucial third element: a stock block is only as trustworthy as the count behind it, and a cashier who can see the menu was refreshed four hours ago knows to trust the shelf over the screen.

The oversold-sale question remains open. The owner had never completed a sale containing oversold lines, so he could not say what the register did at the end of one. That question is therefore *not* answered by this slice and is not claimed to be. What this slice does is make the situation far harder to reach: quantities are clamped at entry, and tender is blocked while a block stands. §7 records what is still unknown.

3. Bug A — root cause

The recon slice proved that all seven places a product can enter the cart funnel through exactly two functions in `src/lib/pos/sale-flow-core.ts`. That is the whole reason this fix is small and safe: there are only two doors, and both are in a pure module.

Both functions clamped the requested quantity to `MAX_LINE_QUANTITY = 99` (`sale-flow-core.ts:199`) and to nothing else. The on-hand count was sitting right there in scope as `product.unitsLeft` and was never read. The register was not failing to *know* the stock; it was declining to *act* on it.

The tender gate had the same shape of omission. `canTender` in `src/app/pos/SaleFlow.tsx` correctly gated on legal category limits, price floors, and High-THC eligibility, and said nothing about stock. Meanwhile `low-stock-core.ts` had already computed the overage and was displaying it as advisory grey text — the register knew it was overselling and only whispered.

One fact from recon is worth restating because it bounds the blast radius: the server-side decrement in `sale-decrement-core.ts` already clamps at zero and reports the oversell rather than writing a negative column. So no database has ever been driven negative by this bug. The real harm was promising a customer product that was not on the shelf, and sending staff to look for it.

4. Bug A — the fix

4.1 A pure core, because the rule is arithmetic

`src/lib/pos/stock-ceiling-core.ts` is new. It holds every decision about what "enough stock" means and it touches no React, no storage, and no network, which is why it can be exhaustively tested.

The single most important design choice in the file is the treatment of an untrustworthy count. `sellableCeiling` (:67) returns a number *only* for a non-negative safe integer. `null`, `undefined`, a negative, a fraction, `NaN`, and `Infinity` all mean "unknown", and `unknown` means unlimited — the old behaviour, exactly. This is what makes the change safe for custom sales, merch, and any untracked item: if we do not have a count we can defend, we do not block a sale on it. A register that refuses to sell because a number was malformed is worse than the bug being fixed.

`isKnownOutOfStock` (:147) is deliberately strict for the same reason: only a *trustworthy* zero greys a tile out.

Around those two predicates sit `clampToStock` (:99), which returns the allowed quantity plus why it was reduced; `canAddOne` (:130) for the tap and `+` paths; `stockRefusalMessage` (:158) for the words the cashier reads; and `stockBlockingLines` (:187) for the tender gate. `clampToStock` takes an `override` flag, so the owner's any-staff override is expressed in the pure layer rather than bolted on in the UI.

The file self-tests via `__runStockCeilingCoreTests` (:208) in the house style, and reports **111 assertions, all passing**.

One of those assertions deserves calling out, because it is the one that would catch a future mistake nobody would otherwise notice. It asserts an invariant between two functions written for different screens: *whatever `clampToStock` allows to enter the cart must never be rejected by `stockBlockingLines` at tender*. It is checked by a nested loop across every combination of stock level and requested quantity in the interesting range. If those two functions ever disagree, the register would allow a cashier to build a cart it then refuses to ring up, with no way forward and no explanation — the worst possible failure at a counter with a customer standing at it. That class of bug is now impossible to introduce silently.

4.2 Enforcement at the two doors

`addToCart` (`sale-flow-core.ts`:223) and `setCartQuantity` (:247) now route every quantity through `clampToStock`. The return type is unchanged (`PosCartEntry[]`), so all seven call sites compiled without modification — which is the entire payoff of the recon step.

Two details are load-bearing. When a clamp means the quantity does not actually change, both functions return the *same array reference* rather than a new one, so React does not re-render and the UI does not flicker on a refused tap. And a known-zero item never creates a cart line at all, rather than creating one and then zeroing it.

`MAX_LINE_QUANTITY = 99` is untouched and its original self-test still passes.

4.3 What the cashier sees

In `src/app/pos/SaleFlow.tsx`:

A zero-stock tile is styled `opacity-45 grayscale border-dashed` and carries `aria-disabled` plus a `data-out-of-stock` hook, driven by `isKnownOutOfStock` at :1834 — and it stays tappable, per the owner's decision. Tapping a tile with nothing left, or pressing `+` past the ceiling, produces the refusal message (:2164 , :2640) naming the product and telling the cashier that "Sell anyway" exists if the unit is genuinely on the shelf.

At the bottom of the cart, `stockBlocks` (:2342) lists every line that exceeds its count, `canTender` (:2357) now includes `stockBlocks.length === 0`, and `menuAgeLabel` (:2369) prints when the menu was last refreshed using the same time format already used elsewhere in the register, guarded against an `Invalid Date`. The **"Sell anyway — the unit is on the shelf"** button sits at :2789 and requires **no PIN** from anyone; while it is active an amber banner states plainly that stock limits are off for this sale, so it can never be left on invisibly.

5. Bug B — root cause

This was not one bug. It was a chain of four entirely reasonable behaviours that combine into an unreasonable outcome, which is why it survived so long.

`SaleFlow` never cleared the cart or the ID verdict when a sale completed, and its snapshot effect kept reporting that finished customer upward to the shell as resumable. Separately, the two-minute idle auto-lock is gated on `screen !== "home"` — but `screen` *stays* "home" for the whole sale, because the `SaleFlow` branch keys off `saleActive`, not `screen`. So the idle timer is live on the "Sale complete" screen. If staff walk away instead of tapping "Done — lock register", that timer fires `lock()`, not `onComplete`, and `lock()` calls `parkActiveSale()` *first*, re-persisting the completed sale. The next PIN entry re-validates an ID which is, of course, still perfectly valid, restores it, and a present verdict starts the flow at `"cart"` rather than `"idgate"`.

Every link is defensible on its own. Together they hand the next customer the last customer's cart and the last customer's age verification.

Critically, the pure resume core was never at fault. A completed sale's ID is a valid ID; nothing in the snapshot distinguishes "parked mid-sale" from "already rung up". No unit test on that module could have caught this, because the module was behaving exactly as designed. The defect lived in the wiring between components — which dictated how it had to be tested (\$6.2).

6. Bug B — the fix

Three independent defences, because a single guard on a four-link chain is a single point of failure.

Defence 1 — a completed sale stops advertising itself. The snapshot effect in `SaleFlow.tsx:506` now short-circuits on `step === "done"` and reports `null`. `step` was added to the dependency array, without which the guard would be dead code. The `onSnapshot` prop type was widened to `... | null` — the doc comment had always promised null was meaningful, but the type never permitted it, which is precisely why a completed sale had no way to say so. The shell honours it at `RegisterShell.tsx:1163`, removing the stored snapshot.

Defence 2 — the shell refuses to park a completed sale. A latch, `saleCompletedRef` (`RegisterShell.tsx:288`), is set in `onComplete` at `:1495` *before* `lock()` is called — ordering that matters absolutely, since `lock()` is what parks. `parkActiveSale` checks the latch at `:894`, before it reads the live sale ref at all, and clears storage instead of writing to it. It is a `useRef` and not `useState` on purpose: `parkActiveSale` also runs from `pagehide` and `visibilitychange` handlers, where a stale closure over state would quietly reintroduce the entire bug.

The latch is released in one central effect at `:861`, keyed on a new sale becoming active. This is the safety valve, and it is the part most likely to be broken by a careless future edit: a latch that never clears would silently destroy session-resume for every subsequent sale, trading this bug for a worse one. It is centralised rather than duplicated across the four `setSaleActive(true)` sites specifically so it cannot be forgotten at a fifth, and a test asserts that exactly one place in the file ever sets it false.

Defence 3 — `lock()` clears the loaded website-order cart too. `lock()` now also calls `setLoadedCart(null)` and `setLoadedMember(null)` (`:940`, `:941`). `SaleFlow`'s `initialCart` falls back through `resumedCart ?? resumeCart ?? loadedCart`; clearing only the first two left the third alive to leak into the next customer's sale.

7. How this was verified

7.1 The RED baseline

A test that has never failed proves nothing about the bug it claims to guard. Both new test files were therefore run against the pre-fix code first.

`tests/compliance/stock-ceiling-core.test.ts` (29 tests) reported **6 failed / 23 passed** before the fix, and the six failures reproduced the owner's report point for point: the second tap on a "1 left" tile was accepted, the `+` button ran away without limit, a huge quantity clamped to 99 instead of to stock, a zero-stock item created a cart line, and the clamp-versus-tender invariant was violated. Several describe blocks are named after his exact words so that if one ever fails again, the failure output says what the owner would say.

`tests/compliance/pos-post-sale-reset.test.ts` (19 tests) reported **13 failed / 6 passed** before the fix. The six that passed are the pure-core tests and the assertions that deliberately pin *correct existing* mechanisms — they were expected to pass both before and after, and they did.

After the fixes: **48 of 48 passing** across both files.

7.2 Why one test file reads source as text

`pos-post-sale-reset.test.ts` reads `RegisterShell.tsx` and `SaleFlow.tsx` as text and asserts the wiring. This is unusual and it is justified: as established in §5, the defect was in how components were connected, and the pure module involved was blameless. A test of the pure module would have passed against the broken register. So this file asserts the things that actually matter and cannot be expressed in a unit test — that the latch is a ref and not state, that it is checked *before* the live ref is read, that `onComplete` latches *before* it calls `lock()`, and that exactly one line in the file releases it.

Writing it surfaced a trap worth recording. The first draft asserted the `onComplete` ordering with `indexOf("lock();")` and reported a **false failure against correct code**, because the explanatory comment above the latch contains the words `lock();` in prose, at an earlier offset than the real call. Prose that quotes code must never be mistaken for code. All ordering assertions now strip whole-line `//` comments first, and the reason is documented in the test file so the next person does not rediscover it the hard way. A second draft failure came from a regex using `[^]*` to match the `onSnapshot` type body, which cannot span the nested brace in `Extract<IdGateVerdict, { allowed: true }>`; that assertion was rewritten to check the declaration structurally.

In both cases the source was verified with `grep` first and the *test* was corrected. Neither assertion was loosened to obtain a green result.

7.3 Full sweep

Typecheck (`npx tsc --noEmit`) is clean. Lint (`npx eslint`) is clean across all six touched files. The full suite is **526 files, 13,387 tests, all passing** — no regressions anywhere in the codebase.

`npx next build` **compiled successfully in 61 seconds**. Its subsequent TypeScript pass was terminated by the sandbox out-of-memory killer, which the kernel log confirms verbatim (`Out of memory: Killed process 25866 (node)`) on a machine with 3.9 GB of RAM. That pass is redundant with the clean `tsc --noEmit` above. The one build warning traces to `accounting/financial-statements`, which this slice does not touch and which warned identically before it.

7.4 What is still unverified

Two things, stated plainly rather than glossed over.

The **oversold-sale completion path** is untested behaviour, because the owner has never produced one and this slice did not build a surfacing report for it. The register now makes oversold carts much harder to create and blocks tender while a block stands, but what happens at the end of a sale where staff used "Sell anyway" is still an open question from recon \$4.

And **on-device confirmation**. Everything above is proven in the test suite and the type system. It has not yet been tapped on the iPad at the counter. That is the next step, and it is the only step that can confirm the greying, the refusal wording, and the post-sale reset behave as intended in the owner's hands.

8. Files changed

New: `src/lib/pos/stock-ceiling-core.ts`, `tests/compliance/stock-ceiling-core.test.ts`, `tests/compliance/pos-post-sale-reset.test.ts`, and this document.

Modified: `src/lib/pos/sale-flow-core.ts` (both mutators honour the ceiling), `src/app/pos/SaleFlow.tsx` (tile state, refusals, tender gate, override, last-refreshed, defence 1), `src/app/pos/RegisterShell.tsx` (defences 2 and 3).

Unchanged by design: `price-override-core`, `scan-required-core`, and every manager-locked path. `low-stock-core` still provides its advisory signal; it is now backed by enforcement instead of standing in for it.